

MODUL MATA KULIAH

TEORI BAHASA OTOMATA

KP181 - 3 SKS



**FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS BUDI LUHUR
JAKARTA**

VERSI 1.1

TIM PENYUSUN

Dr. Mardi Hardjianto S.Kom., M.Kom.
Dr. M. Syafrullah, M.Kom., M.Sc.
Pipin Farida Ariyani, S.Kom., M.T.I.



UNIVERSITAS BUDI LUHUR
FAKULTAS TEKNOLOGI
INFORMASI



MODUL PERKULIAHAN #6

REDUKSI JUMLAH STATE PADA

FINITE STATE AUTOMATA

Capaian Pembelajaran	:	Mahasiswa mengetahui dan memahami konsep membuat FSA yang optimal dengan cara mereduksi jumlah state
Sub Pokok Bahasan	:	6.1. Minimisasi State 6.2. Pengelompokan State 6.3. Tahapan Minimisasi FSA 6.4. Praktikum
Daftar Pustaka	:	1. Utdirartatmo, Firrar. 2005. Teori Bahasa dan Otomata. Graha Ilmu, Yogyakarta

REDUKSI JUMLAH STATE PADA FINITE STATE AUTOMATA

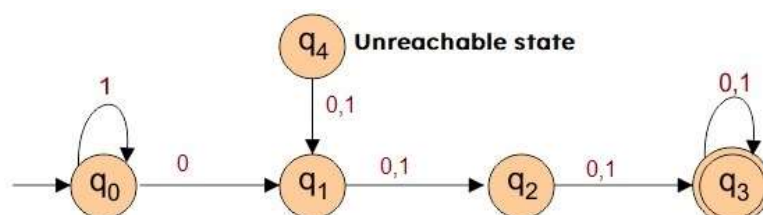
6.1. Minimisasi State

Finite State Automata sesuai dengan namanya merupakan mesin dengan jumlah state terbatas. Ada jumlah minimal state yang diperlukan untuk membuat suatu Finite State Automata menerima suatu bahasa reguler. Untuk sebuah bahasa reguler, kemungkinan ada sejumlah Deterministic Finite Automata yang dapat menerimanya. Perbedaannya hanyalah jumlah state yang dimiliki oleh otomata-otomata yang saling ekuivalen tersebut. Tujuan mereduksi jumlah state (minimisasi) pada Finite State Automata adalah mengurangi jumlah state dari suatu FSA, dengan tidak mengurangi kemampuannya untuk menerima suatu Bahasa. FSA dengan jumlah state yang lebih sedikit merupakan FSA yang paling efisien, biasa disebut juga sebagai Deterministic Finite Automata (DFA) Optimal.

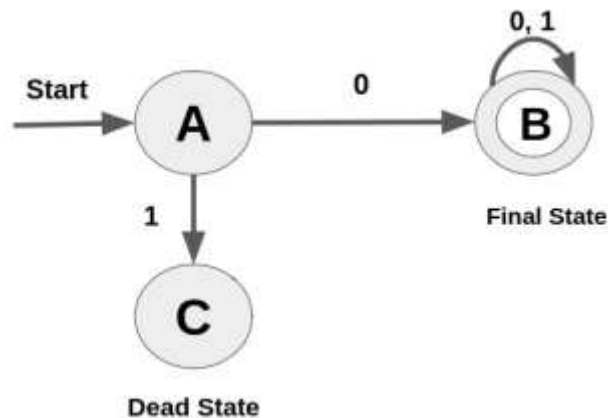
6.2. Pengelompokan State

Sebelum melakukan reduksi FSA, khususnya DFA perlu diketahui sifat dari setiap state pada mesin FSA tersebut. Pengecekan sifat state dapat dilakukan pada single state atau pun pasangan state.

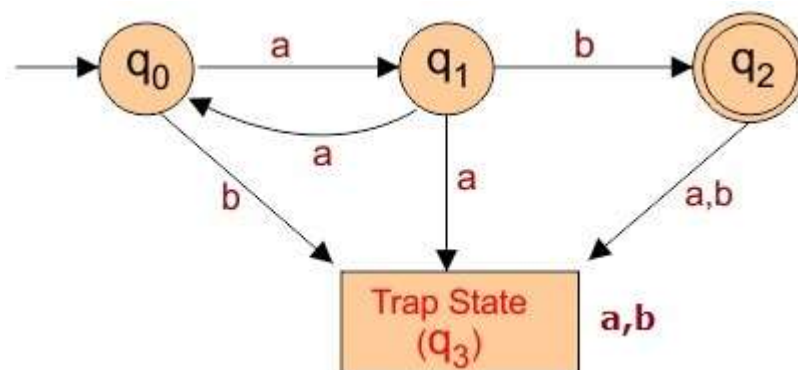
Pada single state bisa bersifat sebagai unreachable state, dead state, mau pun trap state. Unreachable state bisa dikatakan sebagai useless state (tidak berguna) karena state ini tidak bisa dicapai dari awal state dengan input string apa pun. Useless state pada DFA bisa dihapus tanpa mengubah Bahasa yang akan diterima oleh mesin FSA tersebut.



Dead state adalah kondisi dimana tidak ada path (jalan transisi) untuk mencapai kondisi akhir maka kondisi tersebut akan dianggap sebagai dead state. Keadaan tersebut dikenal juga sebagai konfigurasi mati. Dead state dapat dihapus kecuali untuk melengkapi mesin otomatis.



Trap state menganggap input yang diterima sebagai input yang ditolak. Trap state merupakan bagian dari dead state. Konsep ini digunakan dalam DFA, dikenal juga sebagai konfigurasi perangkat. Pada DFA, harus ada path khusus untuk setiap input pada sigma. Jadi beberapa input harus ditolak sesuai dengan kebutuhan. Tidak bisa dihapus karena berguna untuk melengkapi mesin otomatis.



Setiap pasangan *state* pada suatu FSA dapat dikelompokkan menjadi 2 (dua) jenis, yaitu Indistinguishable state dan Distinguishable state. Indistinguishable state adalah pasangan state yang tidak dapat dibedakan (pasangan state dianggap sama). Distinguishable state

adalah pasangan state yang dapat dibedakan (pasangan state dianggap berbeda). Untuk pasangan state yang indistinguishable pada prinsipnya dapat digabungkan menjadi satu state. Reduksi jumlah state dapat dilakukan dengan cara pendekatan ini, yaitu menyatukan pasangan state yang indistinguishable.

a. Indistinguishable State

Pasangan state p dan q dari sebuah FSA dikatakan indistinguishable (tidak dapat dibedakan/sama) jika:

$$\begin{aligned} &\delta(q, w) \in F \text{ begitu pula } \delta(p, w) \in F \\ &\text{dan} \\ &\delta(q, w) \notin F \text{ begitu pula } \delta(p, w) \notin F \\ &\text{untuk semua} \\ &w \in \Sigma^* \end{aligned}$$

Jika w merupakan suatu inputan *string* ($w \in \Sigma^*$), maka transisi *state* (q) terhadap *input* $\{w\}$ merupakan **bagian dari final state** ($\delta(q, w) \in F$) dan transisi *state* (p) terhadap *input* $\{w\}$ merupakan **bagian dari final state** juga ($\delta(p, w) \in F$).

Jika w merupakan suatu inputan *string* ($w \in \Sigma^*$), maka transisi *state* (q) terhadap *input* $\{w\}$ merupakan **bukan bagian dari final state** ($\delta(q, w) \notin F$) dan transisi *state* (p) terhadap *input* $\{w\}$ merupakan **bukan bagian dari final state** juga ($\delta(p, w) \notin F$).

b. Distinguishable state

Dua buah state p dan q dari sebuah FSA dikatakan distinguishable (dapat dibedakan/beda) jika:

$$\begin{aligned} &\text{ada string} \\ &w \in \Sigma^* \\ &\text{sedemikian sehingga} \end{aligned}$$

$\delta(q, w) \in F$ sedangkan $\delta(p, w) \notin F$

Atau

$\delta(q, w) \notin F$ sedangkan $\delta(p, w) \in F$

Jika w merupakan suatu inputan *string* ($w \in \Sigma^*$), maka transisi *state* (q) terhadap *input* $\{w\}$ merupakan bagian dari *final state* ($\delta(q, w) \in F$) dan transisi *state* (p) terhadap *input* $\{w\}$ merupakan bukan bagian dari *final state* ($\delta(p, w) \notin F$).

Jika w merupakan suatu inputan *string* ($w \in \Sigma^*$), maka transisi *state* (q) terhadap *input* $\{w\}$ merupakan bukan bagian dari *final state* ($\delta(q, w) \notin F$) dan transisi *state* (p) terhadap *input* $\{w\}$ merupakan bagian dari *final state* ($\delta(p, w) \in F$).

c. Relasi

Pasangan 2 (dua) state memiliki salah satu kemungkinan, yaitu *distinguishable* atau *indistinguishable* tetapi tidak keduanya. Dalam hal ini terdapat sebuah relasi:

- Jika pasangan state p dan q merupakan *indistinguishable*,
- Dan pasangan state q dan r juga merupakan *indistinguishable*,
- Maka pasangan state p, q , dan r merupakan *indistinguishable*.

Jika *state* $p=q$ dan *state* $q=r$,
maka *state* $p=q=r$.

Dapat disimpulkan bahwa *state* p, q , dan r dapat digabung menjadi 1 (satu) *state*. Dalam melakukan evaluasi state, didefinisikan suatu relasi untuk Q adalah himpunan semua state, yaitu:

- D adalah himpunan state-state *distinguishable*, dimana $D \subset Q$
- N adalah himpunan state-state *indistinguishable*, dimana $N \subset Q$
- Maka $x \in N$ jika $x \in Q$ dan $x \notin D$

6.3. Tahapan Minimisasi FSA

Untuk membuat FSA yang optimal, tahapan minimisasi dilakukan pada mesin DFA. Jika Otomat belum berupa DFA, lebih baik dilakukan perubahan menjadi bentuk DFA terlebih dahulu. Ada beberapa metode untuk melakukan minimisasi pada DFA, diantaranya Equivalence Theorem, Myhill Nerode Theorem (Table Filling Method), dan Partition Method. Pada pembahasan kali ini akan menggunakan Myhill Nerode Theorem.

Teorema Myhill Nerode merupakan teori dasar yang dihasilkan dari teori Bahasa Regular. Teori ini dibuktikan oleh John Myhill dan Anil Nerode pada tahun 1958. Teori ini digunakan untuk membuktikan apakah suatu bahasa L beraturan (regular) atau tidak. Teori ini dapat digunakan untuk meminimalkan keadaan (state) pada DFA. Untuk memahami teorema ini, perlu memahami apa itu Indistinguishability (keadaan yang tidak dapat dibedakan). Berikut tahapan minimisasi menggunakan Teorema Myhill Nerode:

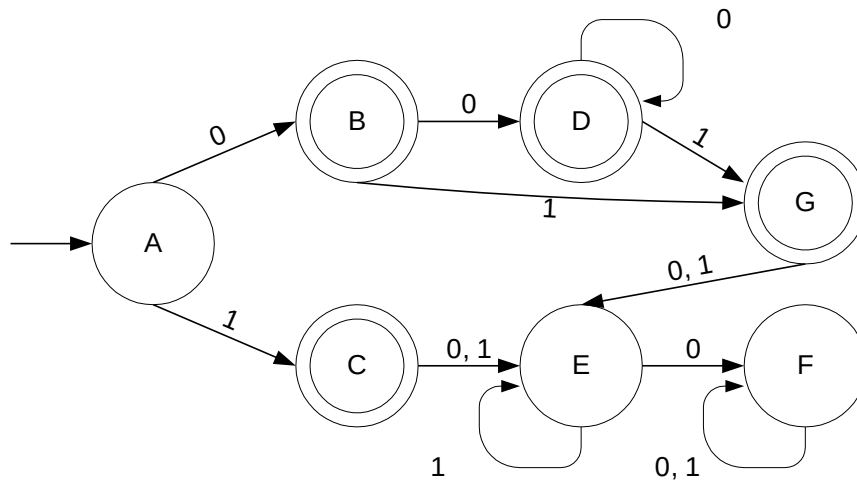
- a. Gambar tabel dengan semua pasangan state $[P, Q]$.
- b. Untuk $[P, Q]$, tandai semua pasangan state final dengan state bukan final ($P \in F$ dan $Q \notin F$).
- c. Jika masih terdapat pasangan $[P, Q]$ yang belum ditandai termasuk bagian dari $[\delta(P, x), \delta(Q, x)]$ yang ditandai maka tandai pasangan $[P, Q]$ tersebut dimana x adalah simbol input. Lakukan berulang (dalam sejumlah pass/level) sampai tidak ada pasangan state yang perlu ditandai.
- d. Gabungan semua pasangan state yang tidak bisa ditandai untuk dijadikan menjadi satu state.

6.4. Praktikum

Berikut beberapa contoh minimisasi DFA menjadi FSA yang optimal.

a. Contoh 1

Diketahui FSA dalam bentuk graph / diagram sebagai berikut:



Tuple dari FSA tersebut adalah:

- $Q = \{A, B, C, D, E, F, G\}$
- $\Sigma = \{0, 1\}$
- $S = \{A\}$
- $F = \{B, C, D, G\}$
- Table Transisi

δ	0	1
A	B	C
B	D	G
C	E	E
D	D	G
E	F	E
F	F	F
G	E	E

Solusi Contoh 1:

❖ Gambar tabel

Buat tabel dengan ukuran sejumlah state. Abaikan pasangan dengan nama state yang sama atau berulang (cell berwarna hitam).

	A	B	C	D	E	F	G
A							
B							
C							
D							
E							
F							
G							

❖ Membuat list pasangan state dari state bukan final dan state final pada FSA.

- $Q \notin F = \{ A, E, F \}$
- $Q \in F = \{ B, C, D, G \}$

Pass 1, terbentuk pasangan state $\{(A, B), (A, C), (A, D), (A, G), (E, B), (E, C), (E, D), (E, G), (F, B), (F, C), (F, D), \text{ dan } (F, G)\}$.

PASS 1

❖ Tandai list pada pass/level 1 (cell berwarna hijau). Beri tanda pasangan state dengan silang (X).

	A	B	C	D	E	F	G
A							
B	X						
C	X						
D	X						
E		X	X	X			
F		X	X	X			
G	X				X	X	

- ❖ Membuat list pasangan state dari state bukan final dan state bukan final pada FSA (sesama bukan final state).
 - $Q \notin F = \{ A, E, F \}$
 - Pasangan state bukan final = $\{(A, E), (A, F), \text{ dan } (E, F)\}$
- ❖ Membuat list pasangan state dari state final dan state final pada FSA (sesama final state).
 - $Q \in F = \{ B, C, D, G \}$
 - Pasangan state final = $\{(B, C), (B, D), (B, G), (C, D), (C, G), \text{ dan } (D, G)\}$

Pass 2, terbentuk pasangan state $\{(A, E), (A, F), (E, F), \{(B, C), (B, D), (B, G), (C, D), (C, G), \text{ dan } (D, G)\}$

PASS 2

- ❖ Lakukan penelusuran pasangan state pada pass/level 2 dengan menggunakan fungsi transisi state terhadap masing-masing inputan.
 - $(A, E): \delta(\{A, E\}, 0) = \delta(A, 0) + \delta(E, 0) = B + F = \{B, F\}$, ditandai karena pasangan state (B, F) termasuk pada list Pass 1.
 - $(A, E): \delta(\{A, E\}, 1) = \delta(A, 1) + \delta(E, 1) = C + E = \{C, E\}$, ditandai karena pasangan state (C, E) termasuk pada list Pass 1.
 - Lakukan untuk semua pasangan state pada Pass 2.

	A	B	C	D	E	F	G
A							
B	X						
C	X	0: (D,E) 1: (E,G)					
D	X	0: (B,D) 1: (G,G)	0: (D,E) 1: (E,G)				
E	0: (B,F) 1: (C,E)	X	X	X			
F	0: (B,F) 1: (C,F)	X	X	X	0: (F,F) 1: (E,F)		
G	X	0: (D,E) 1: (E,G)	0: (E,E) 1: (E,E)	0: (D,E) 1: (E,G)	X	X	

- ❖ Tandai list pada pass/level 2 (cell berwarna kuning). Beri tanda pasangan state dengan silang (X)

	A	B	C	D	E	F	G
A							
B	X						
C	X	X					
D	X	0: (B,D) 1: (G,G)	X				
E	X	X	X	X			
F	X	X	X	X	0: (F,F) 1: (E,F)		
G	X	X	0: (E,E) 1: (E,E)	X	X	X	

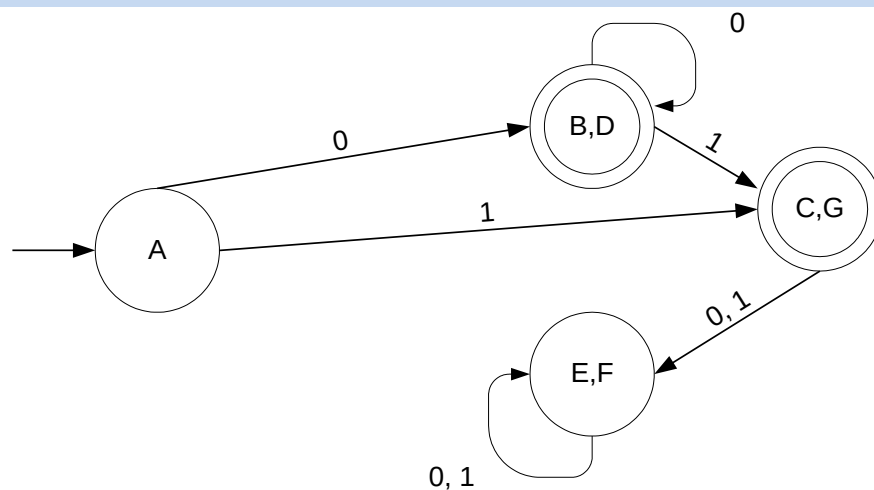
PASS 3

- ❖ Pada level ini tidak ada lagi pasangan state yang dapat ditandai. Sehingga sisa pasangan state {(B,D), (E,F), dan (C,G)}

Sehingga pasangan state yang dapat digabungkan:

- State B digabung dengan state D
- State C digabung dengan state G
- State E digabung dengan state F

Hasil penyederhanaan dalam bentuk FSA adalah sebagai berikut:



b. Contoh 2

Diketahui FSA sebagai berikut:

$Q = \{A, B, C, D, E\}$

$\Sigma = \{0, 1\}$

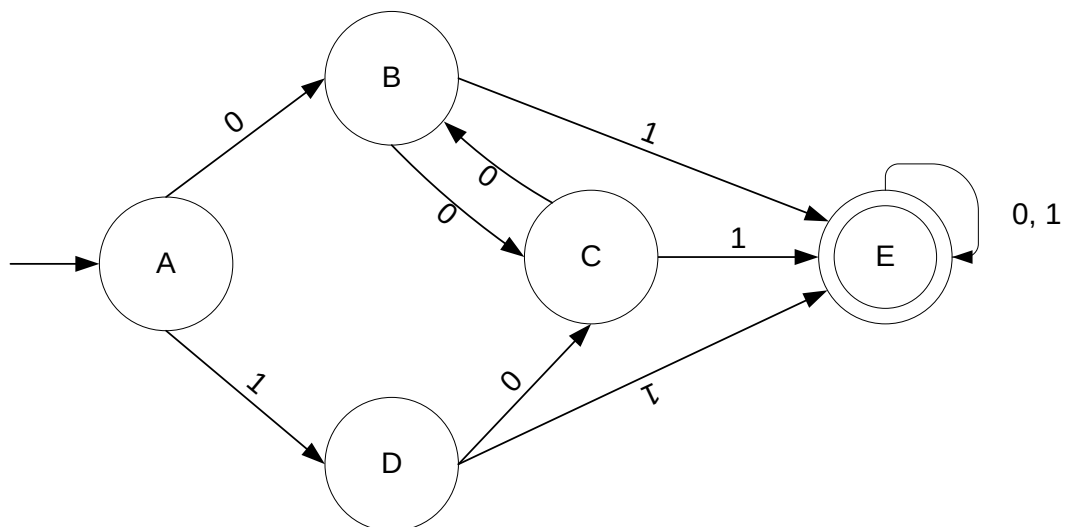
$S = \{A\}$

$F = \{E\}$

- Transisi diberikan dalam tabel berikut:

δ	0	1
A	B	D
B	C	E
C	B	E
D	C	E
E	E	E

- FSA dalam bentuk graph:



Solusi Contoh 2:

B	(B, C)	0			
	(D, E)	1			
C	(B, B)	(C, B)	0		
	(D, E)	(E, E)	1		
D	(B, C)	(C, C)	(B, C)	0	
	(D, E)	(E, E)	(E, E)	1	
E	1	1	1	1	0
	1	1	1	1	1
	A	B	C	D	

Pass 1:

(A, E) (B, E) (C, E) (D, E)

Pass 2:

(A, B) (A, C) (A, D)

Pass 3:

Kosong

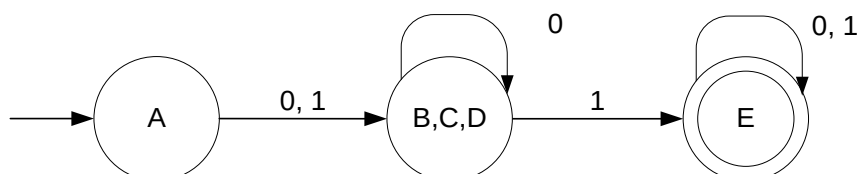
Sisa:

(B, C) (B, D) (C, D)

Sehingga pasangan state yang dapat digabungkan:

- State B dan state C digabung dengan state D

Hasil penyederhanaan dalam bentuk FSA adalah sebagai berikut:



c. **Contoh 3**

Diketahui FSA sebagai berikut:

$Q = \{A, B, C, D, E, F, G, H\}$

$\Sigma = \{0, 1\}$

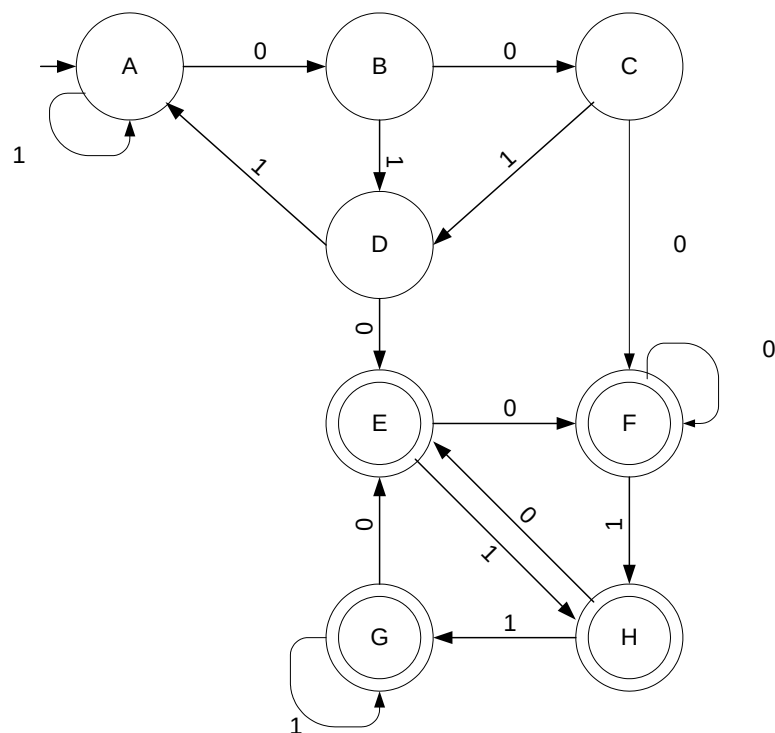
$S = \{A\}$

$F = \{E, F, G, H\}$

- Transisi diberikan dalam tabel berikut:

δ	0	1
A	B	A
B	C	D
C	F	D
D	E	A
E	F	H
F	F	H
G	E	G
H	E	G

- FSA dalam bentuk graph:



Solusi Contoh 3:

B	(B, C)	0						
	(A, D)	1						
C	(B, F)	(C, F)	0					
	(A, D)	(D, D)	1					
D	(B, E)	(C, E)	(F, E)	0				
	(A, A)	(D, A)	(D, A)	1				
E	1	1	1	1	0			
	1	1	1	1	1			
F	1	1	1	1	(F, F)	0		
	1	1	1	1	(H, H)	1		
G	1	1	1	1	(F, E)	(F, E)	0	
	1	1	1	1	(H, G)	(H, G)	1	
H	1	1	1	1	(F, E)	(F, E)	(E, E)	0
	1	1	1	1	(H, G)	(H, G)	(G, G)	1
		A	B	C	D	E	F	G

Pass 1:

(A, E) (A, F) (A, G) (A, H)

(B, E) (B, F) (B, G) (B, H)

(C, E) (C, F) (C, G) (C, H)

(D, E) (D, F) (D, G) (D, H)

Pass 2:

(A, C) (A, D)

(B, C) (B, D)

Pass 3:

(A, B)

(C, D)

Pass 4:

Kosong

Sisa:

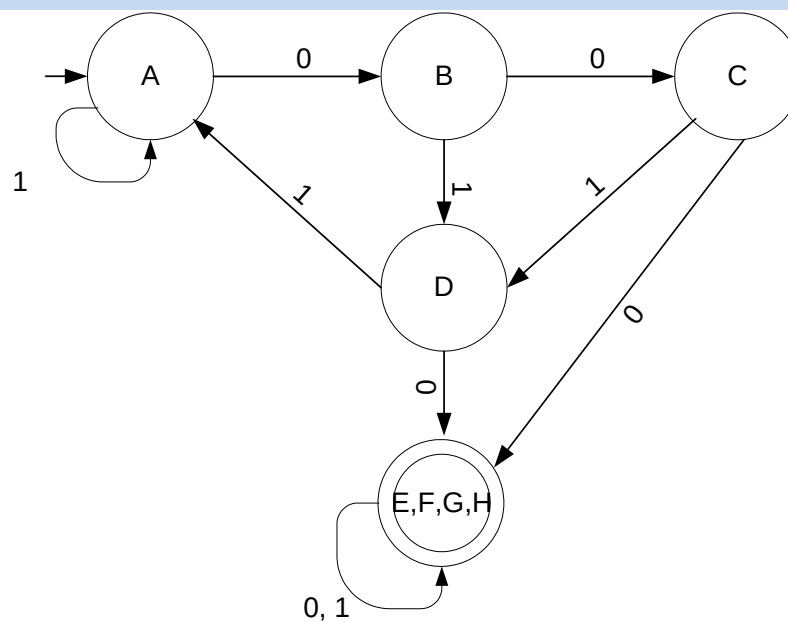
(E, F) (E, G) (E, H)

(F, G) (F, H) (G, H)

Sehingga pasangan state yang dapat digabungkan:

- State E, state F dan state G digabung dengan state H

Hasil penyederhanaan dalam bentuk FSA adalah sebagai berikut:



d. Contoh 4

Diketahui FSA sebagai berikut:

$Q = \{A, B, C, D, E, F, G, H, I, J\}$

$\Sigma = \{a, b\}$

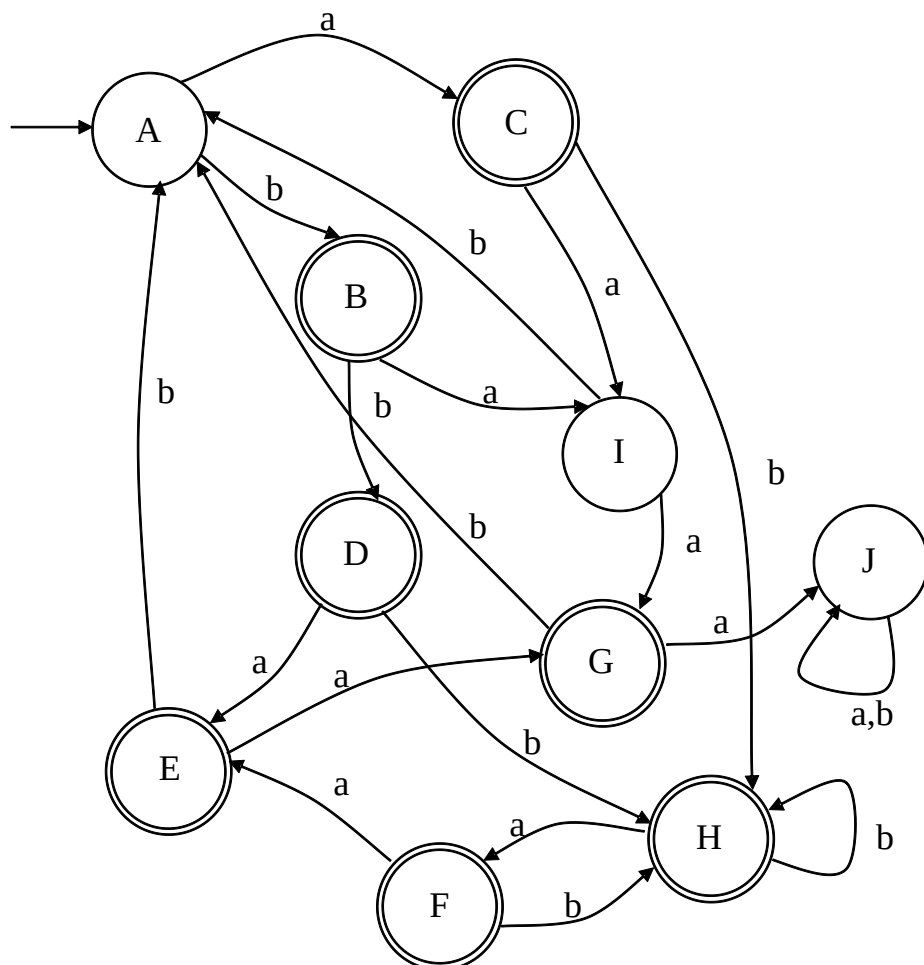
$S = \{A\}$

$F = \{B, C, D, E, F, G, H\}$

- Transisi diberikan dalam tabel berikut:

δ	a	b
A	C	B
B	I	D
C	I	H
D	E	H
E	G	A
F	E	H
G	J	A
H	F	H
I	G	A
J	J	J

- FSA dalam bentuk graph:



Solusi Contoh 4:

B	1	0									
	1	1									
C	1	(I, I)	0								
	1	(D, H)	1								
D	1	(I, E)	(I, E)	0							
	1	(D, H)	(H, H)	1							
E	1	(I, G)	(I, G)	(E, G)	0						
	1	(D, A)	(H, A)	(H, A)	1						
F	1	(I, E)	(I, E)	(E, E)	(G, E)	0					
	1	(D, H)	(H, H)	(H, H)	(A, H)	1					
G	1	(I, J)	(I, J)	(E, J)	(G, J)	(E, J)	0				
	1	(D, A)	(H, A)	(H, A)	(A, A)	(H, A)	1				
H	1	(I, F)	(I, F)	(E, F)	(G, F)	(E, F)	(J, F)	0			
	1	(D, H)	(H, H)	(H, H)	(A, H)	(H, H)	(A, H)	1			
I	(C, G)	1	1	1	1	1	1	1	0		
	(B, A)	1	1	1	1	1	1	1	1		
J	(C, J)	1	1	1	1	1	1	1	(G, J)	0	
	(B, J)	1	1	1	1	1	1	1	(A, J)	1	
	A	B	C	D	E	F	G	H	I		

Pass 1:

(A, B) (A, C) (A, D) (A, E) (A, F) (A, G) (A, H)

(B, I) (B, J)

(C, I) (C, J)

(D, I) (D, J)

(E, I) (E, J)

(F, I) (F, J)

(G, I) (G, J)

(H, I) (H, J)

Pass 2:

(A, I) (A, J)

(B, D) (B, E) (B, F) (B, G) (B, H)

(C, D) (C, E) (C, F) (C, G) (C, H)

(D, E) (D, G)

(E, F) (E, G) (E, H)

(F, G)

(G, H)

(I, J)

Pass 3:

(D, H)

(F, H)

Pass 4:

(B, C)

Pass 5:

Kosong

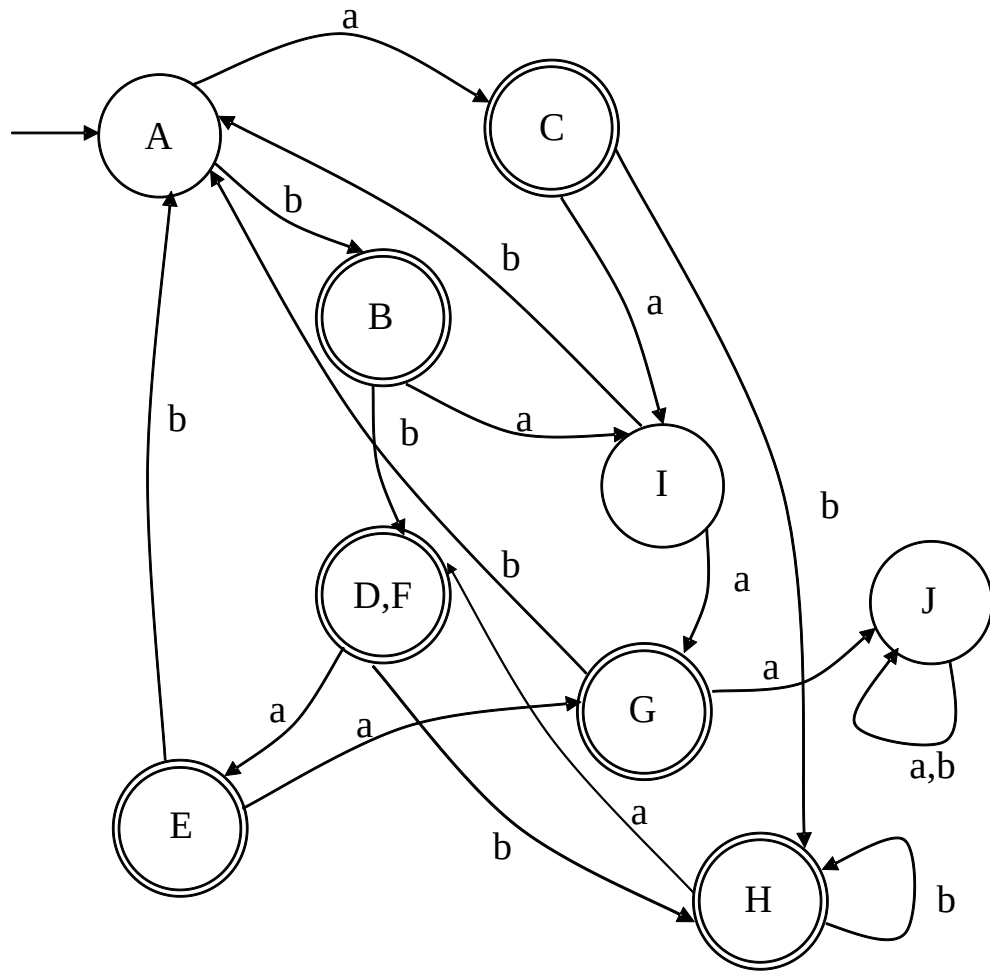
Sisa:

(D, F)

Sehingga pasangan state yang dapat digabungkan:

- State D digabung dengan state F

Hasil penyederhanaan dalam bentuk FSA adalah sebagai berikut:

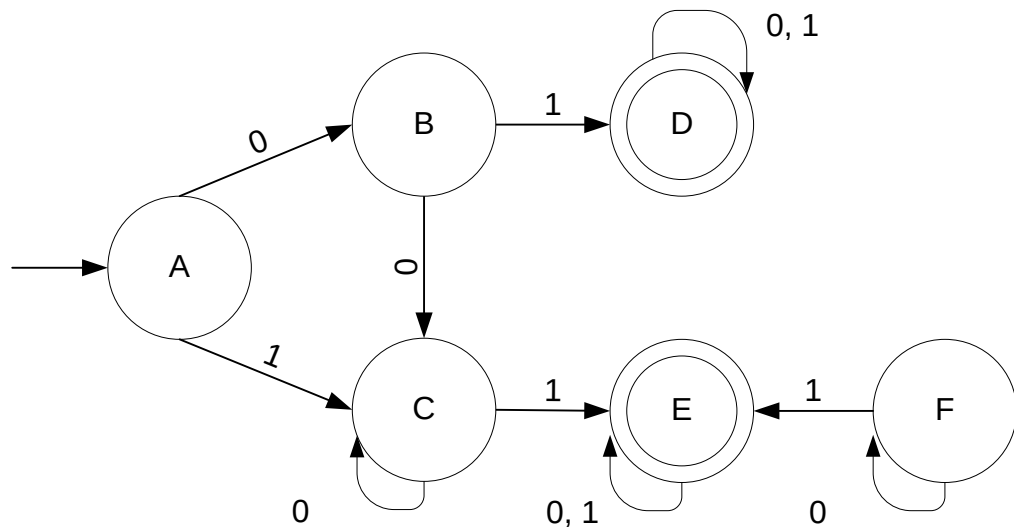


Rangkuman

- Minimisasi jumlah state pada FSA tidak akan mempengaruhi kemampuan mesin tersebut untuk mengenali suatu Bahasa (Ekuivalen).
- FSA dengan jumlah state lebih sedikit akan lebih efisien karena menghemat waktu proses.

Latihan

1. Lakukan reduksi jumlah state pada DFA berikut:



~ Selamat Mengerjakan ~



FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS BUDI LUHUR

Jl. Raya Ciledug, Petukangan Utara, Pesanggrahan

Jakarta Selatan, 12260

Telp: 021-5853753 Fax : 021-5853752

<http://fti.budiluhur.ac.id>